

AlphaEvolve Investment Lab — Key Functions, Comparisons & Benchmarks

TSLA Phase One · generated 2026-09-19 · repo: qizhangumich/stock_investment

1. Key Functions (the load-bearing code)

Function	File	What it does / why it matters
load_market_data()	engine/data.py	Modular data interface (yfinance + CSV cache); swappable for a paid API without touching the rest.
compute_features()	engine/features.py	~40 indicators (returns, trend, momentum, volatility, volume, structure). Trailing windows only - no look-ahead.
classify_regimes()	engine/regimes.py	Labels every day: trend x volatility x bubble, with confidence. Basis for regime gates, matrix, and decision support.
eval_condition() / generate_positions()	engine/genome.py	The strategy interpreter: evaluates JSON rule trees vectorized, then runs the entry/exit state machine. This is the 'DNA reader'.
run_backtest()	engine/backtest.py	Vectorized backtest with 1-day execution lag and 10 bps round-trip costs. The 1-day lag is the core anti-hindsight rule.
compute_metrics()	engine/metrics.py	30+ statistics incl. expectancy = win_rate x avg_win - loss_rate x avg_loss; win rate is never used alone.
Lab.evaluate()	engine/evolution.py	4-stage cascade: cheap screen -> full train -> walk-forward + validation + regimes -> elite qualification. Hidden test computed last, for reporting only.
Lab.fitness()	engine/evolution.py	Composite score: CAGR/Sharpe/Sortino/Calmar + walk-forward robustness + regime stability - turnover - complexity. Weights in config.yaml.
heuristic_mutate() / crossover()	engine/mutation.py	Blind variation: threshold perturbation, indicator swaps, add/remove rules, regime gates, parent recombination.
llm_mutate()	engine/mutation.py	Directed variation: GPT reads genome + performance and proposes a targeted rewrite with an explanation (stored in lineage).
run_generation()	engine/evolution.py	One evolution step: breed ~24 children, evaluate, select, update MAP-Elites archive (family x holding-period grid).
build_daily_report()	engine/report.py	Daily summary: evolution stats, regime, best strategy, population consensus, decision support.

2. What Is Compared Against What

Every evolved strategy is compared against two **fixed, non-evolving benchmarks** (they act as experimental controls and are never mutated): **BENCH_BH** = TSLA Buy & Hold (no costs), and **BENCH_SMA200** = a naive long-above-200-day-average strategy (with costs). Comparisons are always shown per data split, never blended: Train 2012-2019 (in-sample), Validation 2020-2022 (used for selection), Hidden Test 2023-now (out-of-sample, never used for selection or mutation).

Validation 2020-2022	Elite: PriceMomentum_V0267	TSLA Buy & Hold	SMA200 Trend
CAGR	80.1%	62.5%	70.8%
Sharpe	1.53	1.03	1.16
Sortino	1.46	1.52	1.48
Max Drawdown	-37.7%	-73.4%	-60.7%
Win Rate	58.3%	0.0%	26.7%

Profit Factor	6.41	0.00	10.00
Turnover	9.71	0.33	10.00

Hidden Test 2023-2026 (true out-of-sample)	Elite PriceMomentum_V0267	TSLA Buy & Hold	SMA200 Trend
CAGR	24.3%	38.9%	-8.9%
Sharpe	0.87	0.86	-0.02
Sortino	0.72	1.38	-0.02
Max Drawdown	-29.3%	-53.8%	-64.7%
Win Rate	53.3%	0.0%	16.0%
Profit Factor	3.98	0.00	0.64
Turnover	10.38	0.27	13.53

Reading: on the hidden test the elite typically gives up some raw CAGR vs Buy && Hold but wins on risk-adjusted terms (higher Sharpe, roughly half the drawdown) - exactly what the composite fitness optimizes.

3. Mutation Operators Compared (LLM agent vs heuristic)

Two 'agents' create new strategies each generation and compete on equal terms - their children go through the same evaluation cascade: the **heuristic operator** (blind random variation + crossover) and the **LLM operator** (OpenAI gpt-5-mini, directed rewriting; enabled 2026-09-18, ~35% of children).

Operator	Children created	Passed evaluation	Avg fitness (scored)
crossover	97	96	0.563
parameter	67	61	0.558
structural	222	188	0.446
llm	24	21	0.218

Note: the LLM operator only started in generation 17, so its sample is still small; the dashboard's Evolution Growth page tracks whether it pulls ahead over time.

4. Call Graph (text form; see callgraph.mmd for the diagram)

```

run_daily.main()
|- Lab.__init__()
|  |- load_market_data()      data.py    yfinance + cache
|  |- compute_features()      features.py
|  |- classify_regimes()       regimes.py
|- save_benchmarks() -> run_backtest() -> compute_metrics()
|- run_generation() [x N generations]
|  |- load_population()       top 40 by fitness
|  |- breed children:
|  |  |- crossover(a, b)      mutation.py
|  |  |- heuristic_mutate()   mutation.py
|  |  |- llm_mutate()         mutation.py -> OpenAI API
|  |- validate() -> render_code() genome.py
|  |- Lab.evaluate()          4-stage cascade
|  |  |- generate_positions() -> eval_condition() genome.py
|  |  |- run_backtest() -> compute_metrics() stage 1+2
|  |  |- walk_forward() + validation + regime_performance() stage 3
|  |- Lab.fitness()           composite score

```

```
| | '- hidden test (stored only, never selects) stage 4
| | '- _update_archive() MAP-Elites grid
| '- db writes: strategies, lineage, backtests, trades
'- build_daily_report()
  '- population_consensus() -> generate_positions() per strategy

dashboard (Next.js / Vercel)
  '- lib/queries.ts -> node:sqlite (read-only) -> 7 pages
```